

Descoberta automática de elementos acionáveis: uma replicação estendida da abordagem UAES

Sabrina Carlos Costa
Universidade Federal de Campina Grande
Programa de Pós-Graduação em Ciência da Computação
Campina Grande, PB, Brasil
sabrina.costa@copin.ufcg.edu.br

Resumo

A automação de testes de interface gráfica desempenha papel importante na garantia da qualidade de aplicações web modernas, especialmente em abordagens *scriptless* voltadas à geração automática de testes. Entretanto, a identificação automática e não redundante de elementos acionáveis ainda representa um desafio. O estudo original do *Unique Actionable Elements Search* (UAES) apresentou uma abordagem automática para descobrir elementos acionáveis únicos em aplicações web, comparando seus resultados com a abordagem Markup em aplicações open-source e industriais. Este trabalho apresenta uma replicação experimental estendida desse estudo, com foco na recuperação do baseline Markup pelo UAES e em sua variabilidade em um conjunto maior e mais diverso de aplicações open-source. A construção inicial do baseline Markup foi apoiada pelo Codex com GPT-5.5 e revisada manualmente para cada aplicação. A replicação avaliou 20 aplicações, totalizando 587,0 KLOC e 736 testes gerados pelo Cytetion. Na união dos catálogos, foram observados 2.596 elementos, dos quais 900 foram classificados como *same*, 969 como *new* e 727 como *missed*. Considerando o Markup como baseline, o UAES recuperou 55,32% de seus elementos, com intervalo de confiança de 95% entre 39,03% e 68,28%, ficando 24,49 pontos percentuais abaixo dos 79,81% reportados no estudo original para aplicações open-source. Mesmo sem adaptações específicas no UAES para as diferentes tecnologias avaliadas, a abordagem recuperou mais da metade do baseline e identificou 969 candidatos ausentes no Markup. Esses resultados são relevantes diante da heterogeneidade do conjunto e evidenciam o potencial do UAES para complementar a descoberta de elementos, embora a validade funcional dos candidatos adicionais ainda exija confirmação manual sistemática.

Keywords

GUI testing, actionable elements, UAES, systematic GUI testing, experimental replication, large language models

1 Introdução

Em ferramentas automatizadas de teste de interface gráfica para aplicações web, identificar quais componentes da página podem ser acionados é uma etapa essencial antes da geração de testes. Além de reconhecer esses componentes, a ferramenta precisa distingui-los de forma não redundante ao longo da navegação. Essa tarefa é particularmente relevante em interfaces compostas por elementos dinâmicos, componentes reutilizáveis, formulários, menus, botões e links que podem aparecer, desaparecer ou mudar de estado durante a execução da aplicação. Quando essa identificação é incompleta

ou redundante, a exploração automática pode deixar funcionalidades sem teste ou executar interações repetidas sobre elementos já explorados.

A dificuldade de reconhecer esses elementos faz parte de um conjunto mais amplo de desafios da automação de testes GUI. Embora esse tipo de automação busque reduzir o esforço, o custo e o tempo associados à execução manual de testes, a literatura ainda aponta desafios técnicos persistentes, como fragilidade da execução, dificuldade de sincronização entre teste e aplicação, manutenção de scripts e identificação robusta de widgets da interface [5]. Esses desafios afetam diretamente sistemas web modernos, nos quais a interface muda com frequência e os elementos podem ser gerados dinamicamente.

Técnicas de GUI testing *scriptless* buscam reduzir o esforço associado à criação manual de scripts de teste. Nessas abordagens, a ferramenta explora automaticamente a interface da aplicação, identifica elementos interativos e gera sequências de ações a partir dos estados alcançados durante a navegação. A descoberta automática de interfaces já foi apontada como uma etapa relevante para melhorar a geração de testes em aplicações web, pois informações incompletas sobre a interface podem impedir que partes da aplicação sejam exercitadas durante os testes [2]. Assim, a eficácia da exploração depende da capacidade de reconhecer corretamente os elementos acionáveis presentes na GUI, como botões, links, campos de entrada, menus e outros componentes que permitem interação com a aplicação.

Uma estratégia utilizada para apoiar essa identificação é a abordagem baseada em Markup. Nessa abordagem, testadores inserem manualmente anotações no código-fonte da aplicação para indicar quais elementos devem ser considerados durante a exploração automática. Embora essa estratégia auxilie a geração sistemática de testes, ela exige intervenção manual, conhecimento interno da aplicação e manutenção das marcações ao longo da evolução do sistema. Além disso, pode ser limitada em cenários nos quais o código-fonte não está disponível, em componentes de terceiros ou em aplicações com estruturas de interface mais complexas.

Nesse contexto, o Cytetion¹ foi proposto como uma ferramenta de teste sistemático de GUI para aplicações web. A ferramenta adota uma abordagem *scriptless* e progressiva, iniciando a exploração a partir de um caso de teste inicial e gerando novos testes conforme identifica elementos acionáveis em cada estado da interface. O Cytetion também utiliza um oráculo baseado em mensagens da interface, mensagens do console do navegador e códigos de status HTTP para identificar falhas visíveis durante a execução dos testes [3].

¹<https://gitlab.com/lisi-ufcg/cytetion/cytetion>

Para reduzir a dependência de marcações manuais no processo de descoberta de elementos acionáveis, Moura et al. propuseram o *Unique Actionable Elements Search* (UAES)², uma abordagem automática para descoberta única de elementos acionáveis em aplicações web [4]. O UAES utiliza informações presentes no DOM, como texto interno, atributos HTML e propriedades semanticamente expressivas, para construir *locators* capazes de identificar elementos interativos sem exigir modificações no código-fonte da aplicação. A abordagem foi integrada ao Cytetion para apoiar a geração sistemática de testes GUI.

No estudo original, o UAES foi avaliado por meio da comparação com a abordagem Markup em aplicações open-source e industriais. Os elementos identificados foram classificados em três categorias: *same*, quando o elemento foi encontrado tanto pelo UAES quanto pelo Markup; *new*, quando o elemento foi encontrado apenas pelo UAES; e *missed*, quando o elemento foi identificado manualmente, mas não foi encontrado pelo UAES. No estudo com aplicações open-source, os autores avaliaram quatro sistemas e observaram que 73,3% dos elementos acionáveis foram encontrados por ambas as abordagens. Considerando os elementos identificados pelo Markup como baseline, o UAES encontrou automaticamente 79,81% desses elementos. Além disso, 8,1% dos elementos foram identificados apenas pelo UAES e, após revisão manual, foram considerados elementos acionáveis válidos [4].

Apesar dos resultados promissores, o próprio estudo reconhece limitações relacionadas à generalização dos achados para outros conjuntos de aplicações web. Além disso, a eficácia do UAES pode depender da presença de informações semanticamente expressivas no DOM, como textos, identificadores, atributos de acessibilidade e outros valores capazes de diferenciar elementos de forma única. Dessa forma, ainda são necessárias novas evidências sobre o comportamento da abordagem quando aplicada a conjuntos ampliados de aplicações, com diferentes características de interface, navegação e estrutura de componentes.

Além das abordagens baseadas em regras, modelos de linguagem têm sido investigados para compreender a estrutura e o conteúdo de páginas HTML. Gur et al. demonstraram que LLMs ajustados podem classificar semanticamente elementos HTML e selecionar ações de navegação a partir do contexto da página [1]. Esses resultados motivam o uso de um LLM como apoio à identificação e à marcação de elementos acionáveis neste estudo.

Diante desse contexto, este trabalho propõe uma replicação experimental do estudo original. A replicação mantém a comparação entre UAES e Markup e amplia o conjunto de aplicações avaliadas. O objetivo é estimar a capacidade do UAES de recuperar os elementos do baseline Markup e caracterizar a variabilidade desse resultado entre aplicações, preservando as categorias *same*, *new* e *missed*. Com isso, o estudo busca produzir evidências adicionais sobre o comportamento do UAES em um conjunto experimental ampliado.

2 Metodologia

Este trabalho foi conduzido como uma replicação experimental estendida do estudo de Moura et al. [4]. A replicação manteve a comparação entre UAES e Markup e as categorias de classificação

utilizadas no estudo original, mas ampliou o conjunto de aplicações open-source avaliadas e utilizou um LLM como apoio à inserção das marcações, seguida de revisão manual.

O foco desta replicação foi em aplicações open-source. No estudo original, as 20 aplicações industriais foram desenvolvidas com React e a biblioteca de componentes Ant Design, enquanto as quatro aplicações open-source foram caracterizadas pelos autores como sistemas básicos, sem uso de componentes reutilizáveis [4]. Além disso, as aplicações industriais não são igualmente acessíveis para reprodução por terceiros, pois dependem de contexto organizacional, domínio específico e restrições de acesso. Essa limitação dificulta a verificação independente dos resultados e reduz a possibilidade de comparação direta por outros pesquisadores. Assim, este trabalho concentrou a investigação em aplicações open-source diversas, com o objetivo de avaliar a capacidade do UAES de detectar e selecionar elementos acionáveis em um contexto mais acessível, reproduzível e tecnologicamente heterogêneo.

Dessa forma, o estudo estimou a recuperação do baseline Markup pelo UAES quando aplicado a um conjunto ampliado de aplicações open-source e caracterizou a variação dessa medida entre aplicações. A comparação foi realizada a partir das mesmas categorias utilizadas pelos autores: *same*, *new* e *missed*. As 20 aplicações foram tratadas conjuntamente na análise principal. O algoritmo, os critérios de descoberta e as regras de construção de locators do UAES não foram modificados ou ajustados para tecnologias específicas das aplicações avaliadas.

2.1 Problema de pesquisa

Este trabalho investigou a capacidade do UAES de recuperar elementos do baseline Markup e a variabilidade dessa capacidade quando aplicado a um conjunto ampliado de aplicações web open-source. O estudo original apresentou resultados positivos, mas as aplicações industriais utilizadas não são igualmente acessíveis para reprodução por terceiros. Assim, tornou-se necessário avaliar experimentalmente o comportamento do UAES em um conjunto mais amplo e aberto de aplicações.

Esse problema é relevante porque ferramentas de teste *scriptless* dependem da identificação correta dos elementos acionáveis para explorar a interface de forma sistemática. Caso a abordagem automática não consiga identificar elementos relevantes ou gere muitos elementos redundantes, a qualidade da exploração pode ser comprometida. Além disso, avaliar a técnica em aplicações open-source favorece a reprodutibilidade do experimento e permite que os resultados sejam verificados em estudos futuros.

2.2 Questão de pesquisa

O experimento foi orientado pela seguinte questão de pesquisa:

- **QP:** Qual é a capacidade do UAES de recuperar os elementos acionáveis presentes no baseline Markup e como essa capacidade varia entre as 20 aplicações web open-source avaliadas?

Neste trabalho, a capacidade do UAES de auxiliar na detecção e seleção de elementos acionáveis foi avaliada pela comparação entre os elementos identificados automaticamente pelo UAES e os elementos presentes no conjunto Markup. Essa comparação seguiu as

²<https://gitlab.com/lisi-ufcg/cytetion/loc-study/execute-study-uaes>

categorias utilizadas no estudo original: *same*, *new* e *missed*. O principal indicador analisado foi o percentual de elementos do Markup encontrados pelo UAES, calculado por: $same/(same + missed)$.

2.3 Hipóteses

A partir da questão de pesquisa, foram definidas as seguintes hipóteses:

- **H0:** A taxa de recuperação do baseline Markup pelo UAES no conjunto avaliado é igual ao valor de referência de 79,81% reportado no estudo original para aplicações open-source.
- **H1:** A taxa de recuperação do baseline Markup pelo UAES no conjunto avaliado difere do valor de referência de 79,81%.

A capacidade do UAES foi operacionalizada pela taxa de recuperação do baseline, definida por $\theta = same/(same + missed)$. O percentual de 79,81% reportado no estudo original para aplicações open-source foi empregado exclusivamente como referência histórica contextual.

2.4 Design do experimento

O estudo teve caráter experimental e comparativo e adotou um desenho pareado e bloqueado por aplicação. A abordagem de descoberta, UAES ou Markup, constituiu o fator experimental, enquanto cada aplicação foi tratada como unidade experimental e bloco. A abordagem Markup foi usada como baseline, preservando o papel atribuído a ela no desenho experimental original. Nesta replicação, esse conjunto foi construído inicialmente com apoio do Codex e submetido a revisão manual. As duas abordagens foram executadas simultaneamente pelo Cyttestion sobre os mesmos estados da interface, reduzindo diferenças decorrentes de alterações dinâmicas no DOM ou de caminhos distintos de exploração.

A variável independente foi a abordagem de descoberta de elementos, com os níveis UAES e Markup. As 20 aplicações constituíram as unidades experimentais e o contexto no qual as abordagens foram comparadas. Foi realizada uma execução pareada por aplicação. Não houve alocação aleatória entre abordagens, pois ambas foram executadas simultaneamente no mesmo fluxo de exploração. A ausência de repetições impede estimar a variabilidade entre execuções e é tratada como ameaça à validade.

As variáveis dependentes principais do experimento são:

- quantidade de elementos classificados como *same*;
- quantidade de elementos classificados como *new*;
- quantidade de elementos classificados como *missed*;
- taxa de recuperação do baseline Markup, calculada por $same/(same + missed)$.

Essas categorias seguem a classificação utilizada no estudo original. Elementos *same* são aqueles encontrados tanto pelo UAES quanto pela abordagem Markup. Elementos *new* são aqueles encontrados apenas pelo UAES. Elementos *missed* são aqueles identificados pela abordagem Markup, mas não encontrados pelo UAES.

Além dessas variáveis, foram calculados totais derivados para apoiar a análise. O total de elementos identificados pelo Markup foi definido como *same + missed*. O total de elementos identificados pelo UAES foi definido como *same + new*. A união dos elementos observados na comparação foi definida como *same + new + missed*. Como os elementos *new* não foram submetidos a validação manual

sistemática, essa união não foi interpretada como um conjunto completo de elementos acionáveis válidos.

2.5 Dados

Os dados do experimento foram compostos pelos elementos acionáveis identificados em cada aplicação web avaliada. Esses dados foram coletados durante a execução da abordagem Markup e da abordagem UAES. A abordagem Markup produziu o conjunto de elementos marcados no código-fonte com apoio do Codex e revisão manual, enquanto o UAES produziu o conjunto de elementos identificados automaticamente. A comparação entre esses conjuntos permitiu classificar os elementos nas categorias *same*, *new* e *missed*.

O conjunto experimental foi formado por aplicações web open-source que puderam ser executadas em ambiente controlado. Inicialmente, foram consideradas as aplicações open-source avaliadas no estudo original: Petclinic³, Bistro Restaurant⁴, Learn Educational⁵ e School Educational⁶. Em seguida, foram adicionadas novas aplicações abertas com diferentes características de interface, navegação e interação, de acordo com a viabilidade de instalação, execução e análise.

As novas aplicações foram selecionadas com base nos seguintes critérios: (i) código-fonte público; (ii) possibilidade de execução local ou em ambiente controlado; (iii) presença de interface web navegável; (iv) presença de elementos acionáveis variados, como botões, links, formulários e menus; e (v) documentação mínima de instalação e execução.

Os dados foram armazenados em formato tabular, preferencialmente CSV ou JSON. Para cada elemento, foram registrados a aplicação, a página ou estado da interface, o tipo do elemento, o *locator* identificado, a abordagem que encontrou o elemento e a classificação final. Não há previsão de uso de dados sensíveis de usuários, pois o estudo analisa elementos de interface das aplicações avaliadas.

2.6 Ferramentas e ambiente

O experimento utilizou o UAES integrado ao Cyttestion. O Cyttestion foi responsável pela exploração sistemática da interface e pela geração dos testes durante a navegação. Para apoiar a construção inicial do baseline Markup, foi utilizado o Codex com GPT-5.5 na análise do código-fonte das aplicações e na inserção de atributos data-cy nos elementos classificados como acionáveis. O modelo recebeu instruções explícitas sobre a definição de acionabilidade e sobre os critérios de unicidade, estabilidade e expressividade semântica dos identificadores. As marcações produzidas foram posteriormente revisadas manualmente antes da execução experimental. O Codex foi empregado como instrumento de apoio, e não como avaliador ou tratamento experimental independente.

As execuções experimentais foram realizadas em uma máquina com processador 13th Gen Intel® Core™ i5-13420H, 32,0 GiB de memória RAM e sistema operacional Ubuntu 22.04.5 LTS de 64 bits

³<https://github.com/spring-projects/spring-petclinic>

⁴<https://github.com/Sathish2905/sample-website-templates/tree/master/bestro-restaurant-bootstrap-html5-template>

⁵<https://github.com/Sathish2905/sample-website-templates/tree/master/learn-educational-free-responsive-web-template>

⁶<https://github.com/Sathish2905/sample-website-templates/tree/master/school-educational-html5-template>

e o ambiente de execução incluiu Node.js⁷, Cypress⁸, navegador web e as dependências necessárias para executar cada aplicação selecionada.

O conjunto de instruções fornecido ao Codex foi preservado como parte dos artefatos do estudo. A documentação completa das versões do UAES, Cyttestion, Node.js, Cypress, navegador e de cada aplicação avaliada ainda deverá ser incorporada ao pacote de replicação para permitir a reprodução integral do ambiente experimental.

2.7 Procedimento experimental

Como exibido na Figura 1, o experimento foi executado em seis etapas. Inicialmente, as aplicações foram selecionadas, configuradas e executadas localmente em ambiente controlado e documentado, com definição de uma URL inicial para exploração.

Em seguida, foi realizada a marcação inicial dos elementos acionáveis com apoio do Codex com GPT-5.5. O modelo analisou o código-fonte do frontend e inseriu atributos data-cy de acordo com os critérios de acionabilidade, unicidade, estabilidade e expressividade semântica definidos previamente. As marcações produzidas foram revisadas manualmente.

Na terceira etapa, o UAES integrado ao Cyttestion foi executado simultaneamente à abordagem Markup sobre cada aplicação para identificar os elementos disponíveis nos mesmos estados explorados. Na quarta etapa, os elementos identificados pelas duas abordagens foram comparados e classificados como *same*, *new* e *missed*. Na quinta etapa, os elementos *new* e *missed* foram analisados de forma preliminar para apoiar a interpretação dos resultados e identificar possíveis causas das divergências entre UAES e Markup. A validação manual sistemática desses elementos foi tratada como etapa complementar e indicada como trabalho futuro.

Por fim, os dados foram consolidados e analisados por aplicação e de forma agregada, permitindo a comparação com os resultados reportados no estudo original. Cada elemento foi classificado em uma das seguintes categorias:

- **same:** elemento encontrado tanto pelo UAES quanto pelo Markup;
- **new:** elemento encontrado apenas pelo UAES;
- **missed:** elemento encontrado pelo Markup, mas não identificado pelo UAES.

2.8 Métricas

As métricas analisadas foram a quantidade de elementos classificados como *same*, *new* e *missed*, o percentual de elementos do Markup encontrados pelo UAES e o percentual de elementos adicionais encontrados pelo UAES.

O percentual de elementos do Markup encontrados pelo UAES foi calculado pela razão entre os elementos classificados como *same* e o total de elementos identificados pela abordagem Markup:

$$\text{Percentual Markup encontrado} = \frac{\text{same}}{\text{same} + \text{missed}}$$

Essa métrica indica a proporção do baseline Markup que foi identificada automaticamente pelo UAES.

O percentual de candidatos adicionais encontrados pelo UAES foi analisado a partir dos elementos classificados como *new*. Esses elementos correspondem aos elementos identificados automaticamente pelo UAES que não foram identificados pela abordagem Markup. Quando reportado em percentual, esse valor foi calculado em relação à união dos elementos observados na comparação:

$$\text{Percentual de adicionais} = \frac{\text{new}}{\text{same} + \text{new} + \text{missed}}$$

Também foram analisadas as quantidades absolutas de elementos *same*, *new* e *missed* por aplicação. Essa análise permitiu observar o comportamento do UAES em cada aplicação e de forma agregada no conjunto ampliado de aplicações open-source.

2.9 Análise dos dados

A análise foi feita inicialmente por estatística descritiva. Foram calculados totais, médias, porcentagens e distribuição dos elementos nas categorias *same*, *new* e *missed*. A métrica principal foi o percentual de elementos do Markup encontrados pelo UAES, calculado por $\text{same}/(\text{same} + \text{missed})$.

O resultado obtido nas 20 aplicações foi contextualizado pelo valor de 79,81% reportado no estudo original para aplicações open-source. Essa estimativa foi tratada como referência histórica fixa; não foi definida margem de similaridade ou equivalência.

Além da métrica principal, os elementos classificados como *new* e *missed* foram analisados para apoiar a interpretação dos resultados. Essa análise buscou identificar indícios de elementos adicionais e possíveis causas para elementos não encontrados pelo UAES, como ausência de atributos semanticamente distinguíveis, reutilização de identificadores, presença de locators iguais ou comportamento dinâmico da interface. A confirmação funcional sistemática desses elementos é discutida como trabalho futuro.

Para estimar a incerteza da taxa agregada, foi aplicado bootstrap não paramétrico agrupado por aplicação, com 100.000 reamostragens e semente 20260716. Em cada reamostragem foram selecionadas 20 aplicações com reposição e recalculada a razão entre a soma dos elementos *same* e a soma do baseline *same* + *missed*. Essa estratégia preserva a dependência entre elementos pertencentes à mesma aplicação e evita tratá-los indevidamente como observações independentes. Também foram calculadas a média, a mediana, o desvio-padrão, o intervalo interquartil e a amplitude das taxas por aplicação. Não foi assumida normalidade dessas taxas; dada a natureza limitada da variável e o número de unidades experimentais, optou-se por estatística descritiva robusta e bootstrap não paramétrico, que não depende dessa suposição.

Para operacionalizar a avaliação das hipóteses, adotou-se a seguinte regra inferencial: H0 foi considerada não sustentada quando o valor histórico de 79,81% ficou fora do intervalo de confiança bootstrap de 95% da taxa agregada; caso o valor estivesse contido no intervalo, H0 não seria rejeitada. Essa regra trata o resultado original como referência histórica fixa e não constitui um teste de equivalência nem uma comparação que incorpore a incerteza da estimativa original.

⁷<https://nodejs.org>

⁸<https://www.cypress.io/>

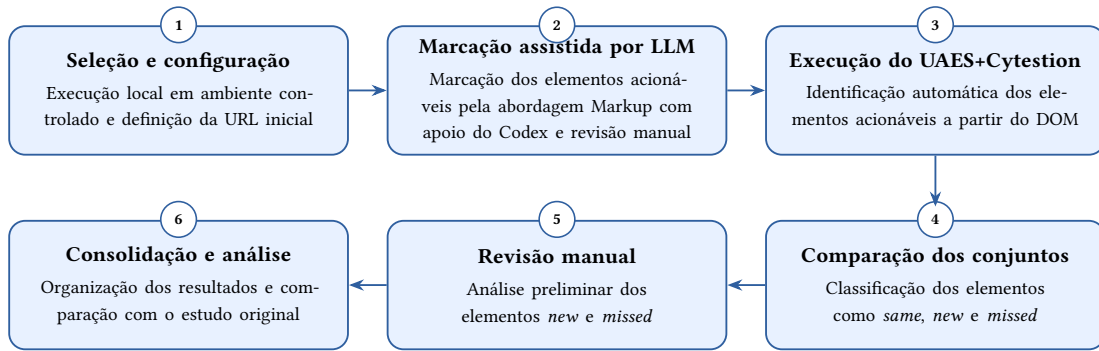


Figura 1: Fluxo do procedimento experimental da replicação.

2.10 Reprodutibilidade

Os artefatos atualmente disponibilizados foram organizados em um repositório público.⁹ O repositório reúne a lista das aplicações avaliadas, scripts de análise, arquivos de saída, dados em CSV ou JSON, o conjunto de instruções utilizado com o Codex e a classificação dos elementos em *same*, *new* e *missed*. No estado atual, esses artefatos permitem reproduzir a análise a partir dos catálogos coletados. A documentação das versões das ferramentas, das revisões exatas das aplicações e da configuração completa de execução deverá ser acrescentada para viabilizar a reprodução integral desde a preparação do ambiente e a nova coleta dos catálogos.

3 Resultados e Discussão

A replicação avaliou 20 aplicações web open-source, comparando a abordagem automática UAES com a abordagem Markup. Assim como no estudo original, os elementos acionáveis identificados foram classificados em três categorias: *same*, quando encontrados por ambas as abordagens; *new*, quando encontrados apenas pelo UAES; e *missed*, quando encontrados pelo Markup, mas não pelo UAES.

A Tabela 1 apresenta as aplicações avaliadas, seus respectivos tamanhos em KLOC e a quantidade de testes gerados pelo Cytetion em cada projeto. Ao todo, o conjunto experimental possui 587,0 KLOC e resultou na geração de 736 testes.

Diferentemente do estudo original, cujo conjunto open-source era composto majoritariamente por aplicações mais simples e estruturalmente semelhantes, esta replicação avaliou aplicações com características mais heterogêneas. O conjunto inclui projetos com diferentes tamanhos, stacks tecnológicas e estilos de implementação, como HTML/CSS/JavaScript tradicional, Spring Boot, React, React com Vite, React com TypeScript, Next.js, Vue com Bootstrap, Angular/TypeScript, Tailwind CSS, Material UI, Ant Design, Redux e styled-components.

Aplicações modernas baseadas em componentes podem apresentar estruturas DOM mais profundas, elementos encapsulados, textos repetidos, classes dinâmicas e menor presença de atributos semanticamente distinguíveis. Como o UAES depende das informações presentes no DOM para identificar e diferenciar elementos acionáveis, essas características constituem hipóteses explicativas

Tabela 1: Aplicações avaliadas, tamanho em KLOC e testes gerados.

Projeto	KLOC	Testes
bistro-restaurant	12,3	118
cv-application	39,8	7
cypress-realworld-app	36,7	47
feedback-app	39,4	4
habit-tracker-web-app	4,6	10
job-tracker	3,9	1
juice-shop	142,9	16
learn-educational	5,1	112
open-resume	44,9	34
petclinic	20,7	141
pizzaflo	11,1	11
react-hotel-booking-app	31,2	18
react-hotel-reservation-system	47,8	4
react-restaurant	19,0	16
react-shopping-cart	30,1	9
refine-dashboard	28,5	36
restaurant-react-js-app	5,0	16
school-educational	11,0	121
todomvc	17,4	6
vue-crud-bootstrap	35,6	9
Total	587,0	736

para a variação entre aplicações, mas seus efeitos não foram isolados experimentalmente neste estudo.

Na união dos catálogos, foram observados 2.596 elementos. Nos artefatos gerados pelos scripts de comparação, os rótulos *equal*, *minus* e *plus* correspondem, respectivamente, às categorias *same*, *missed* e *new* adotadas no texto. Desse conjunto, 900 foram classificados como *same*, 969 como *new* e 727 como *missed*. O baseline Markup continha 1.627 elementos, dos quais 900 também foram encontrados pelo UAES, resultando em taxa agregada de recuperação de 55,32%. Além disso, 37,33% dos elementos observados foram classificados como *new*. Esses 969 elementos representam candidatos encontrados somente pelo UAES e não devem ser interpretados como elementos acionáveis válidos sem confirmação manual sistemática.

A Figura 2 sintetiza a distribuição geral dos elementos classificados. Observa-se que o número de elementos *new* foi superior ao número de elementos *same*. O UAES recuperou pouco mais da metade do baseline Markup e também identificou uma quantidade expressiva de candidatos que não estavam presentes nesse baseline.

⁹<https://github.com/csabrira/rpc-uaes-experimental-replication>

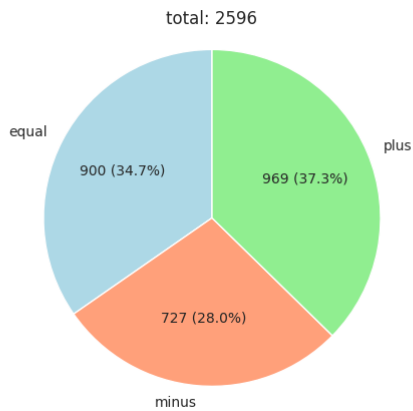


Figura 2: Distribuição total dos elementos classificados no conjunto de aplicações avaliadas.

A interpretação funcional desses candidatos, entretanto, depende de validação manual sistemática.

A Figura 3 permite observar a variação na composição das três categorias, usando a união como denominador. Considerando separadamente a recuperação do baseline, calculada por $\text{same}/(\text{same} + \text{missed})$, projetos como *job-tracker*, *todomvc* e *react-hotel-reservation-system* apresentaram 100%. Por outro lado, aplicações como *refine_dashboard*, *restaurant-react-js-app*, *react-shopping-cart*, *juice-shop* e *open-resume* apresentaram percentuais baixos. A taxa média por aplicação foi de 63,74%, com mediana de 70,51%, desvio-padrão de 29,26 pontos percentuais, intervalo interquartil de 40,41% a 85,13% e valores entre 10,83% e 100%. Essa amplitude mostra que a recuperação do baseline variou substancialmente entre as aplicações avaliadas.

O intervalo de confiança de 95% da taxa agregada, estimado por bootstrap agrupado por aplicação, foi de 39,03% a 68,28%. A estimativa ficou 24,49 pontos percentuais abaixo dos 79,81% reportados no estudo original, e essa referência histórica não esteve contida no intervalo estimado para a replicação. Tomando 79,81% como referência fixa, os dados não sustentaram H_0 e forneceram evidência de que o resultado obtido no conjunto avaliado difere do valor original. Essa comparação não incorpora a incerteza da estimativa do estudo original, cujos dados por aplicação não foram utilizados em uma comparação inferencial independente.

O UAES identificou 969 elementos *new*, isto é, candidatos detectados automaticamente que não estavam presentes no conjunto Markup. Embora a ausência de validação manual sistemática impeça afirmar que todos sejam acionáveis ou representem ganho funcional, esse volume reforça o potencial do UAES para detectar elementos que a abordagem Markup pode não contemplar. O resultado é consistente com o papel complementar observado no estudo original, no qual a inspeção manual confirmou elementos válidos encontrados somente pelo UAES.

Apesar de a taxa agregada ter ficado abaixo da referência histórica, o resultado permanece relevante porque foi obtido sem modificações no UAES para acomodar as diferentes tecnologias do

conjunto. A implementação original foi aplicada a projetos com stacks, bibliotecas de componentes, tamanhos e estruturas de DOM heterogêneos, enquanto o estudo original avaliou aplicações industriais uniformemente baseadas em React e Ant Design e aplicações open-source mais simples. Nesse cenário mais variado, recuperar mais da metade do baseline Markup e alcançar taxas elevadas em parte das aplicações constitui evidência promissora de aplicabilidade fora do contexto tecnológico predominante na avaliação original. Essa interpretação não elimina a variabilidade observada nem implica generalização para toda aplicação web, mas indica que a abordagem preserva utilidade potencial sem ajustes específicos por tecnologia.

Os principais achados desta replicação são: o UAES recuperou 55,32% do baseline Markup no conjunto das 20 aplicações; essa estimativa ficou abaixo da referência histórica do estudo original; as taxas apresentaram elevada variabilidade entre aplicações; e, sem receber adaptações específicas para as tecnologias avaliadas, o UAES identificou um volume expressivo de candidatos adicionais, reforçando seu potencial como apoio complementar à descoberta de elementos.

4 Ameaças à Validade

Uma ameaça à validade de construção está relacionada à métrica principal utilizada neste estudo. A taxa $\text{same}/(\text{same} + \text{missed})$ mede a proporção dos elementos identificados pela abordagem Markup que também foram encontrados pelo UAES. Essa métrica é adequada para comparar a descoberta automática com o baseline Markup, mas não mede diretamente cobertura de código, cobertura funcional, detecção de falhas ou qualidade final dos testes gerados. Assim, a descoberta de elementos acionáveis deve ser interpretada como uma condição necessária para a exploração sistemática da interface, mas não como uma medida completa da efetividade da suíte de testes.

Outra ameaça está relacionada ao uso da abordagem Markup como baseline. Embora essa escolha siga o desenho experimental do estudo original, no trabalho original as marcações foram produzidas por testadores experientes, enquanto nesta replicação a construção inicial foi apoiada pelo Codex com GPT-5.5 e posteriormente revisada manualmente. O modelo foi utilizado como instrumento de apoio, mas não houve avaliação independente da completude do baseline produzido.

O baseline pode ser afetado tanto por omissões, inconsistências e decisões subjetivas durante a revisão humana quanto por variações, classificações incorretas ou elementos não identificados pelo modelo e pela revisão manual. Consequentemente, elementos classificados como *new* podem corresponder a elementos acionáveis válidos que não foram incluídos no Markup, enquanto elementos *missed* podem refletir limitações do UAES ou particularidades do processo de marcação assistida. Para reduzir essa ameaça, foi utilizado o mesmo conjunto de instruções para todas as aplicações, seguido de revisão manual das marcações produzidas. Ainda assim, o baseline não deve ser considerado necessariamente completo ou livre de erros.

Há também ameaças relacionadas ao ambiente de execução das aplicações. Aplicações web modernas podem depender de carregamento assíncrono, estado inicial específico, dados pré-carregados,

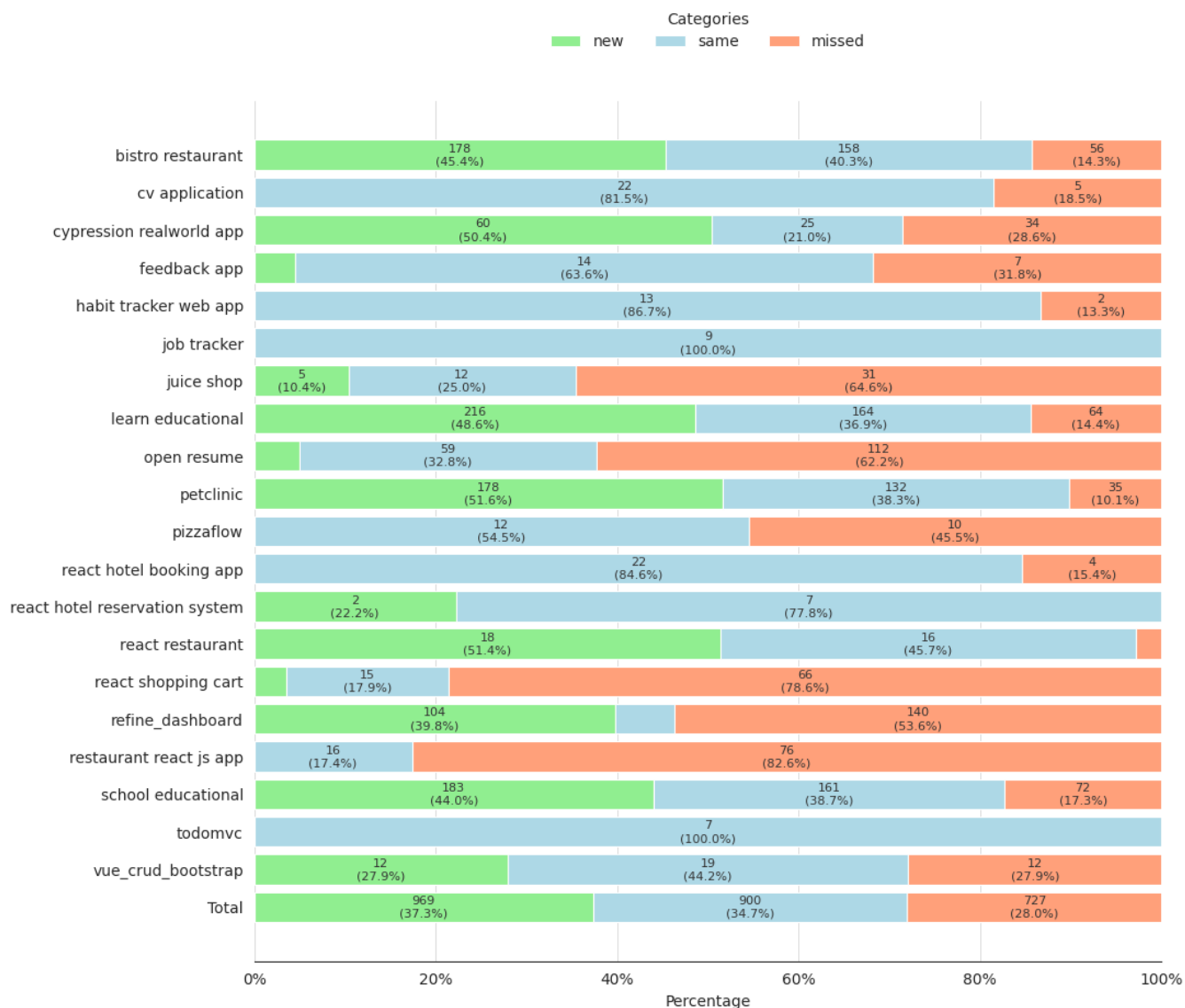


Figura 3: Distribuição percentual dos elementos *same*, *new* e *missed* por aplicação open-source, usando a união das três categorias como denominador.

serviços externos, diferenças de navegador e variações de renderização. Esses fatores podem alterar os elementos disponíveis durante a exploração e, consequentemente, influenciar os catálogos gerados. Para reduzir esse risco, a execução foi realizada em ambiente controlado e as duas abordagens foram executadas simultaneamente, mas ainda assim pequenas variações de ambiente podem afetar os resultados.

Foi realizada uma única execução pareada por aplicação. Embora a execução simultânea reduza diferenças entre UAES e Markup dentro de cada aplicação, a ausência de repetições não permite estimar a variabilidade entre execuções do Cytetion. Assim, os intervalos apresentados caracterizam a variação entre aplicações, e

não a estabilidade dos resultados em múltiplas execuções de uma mesma aplicação.

A validade externa é limitada ao conjunto de aplicações avaliadas. Embora esta replicação tenha ampliado o número de aplicações open-source em relação ao estudo original, os resultados não devem ser generalizados automaticamente para todos os tipos de aplicações web. O conjunto avaliado inclui aplicações com diferentes tamanhos, stacks tecnológicas e estilos de implementação, como HTML/CSS/JavaScript tradicional, Spring Boot, React, Next.js, Vue, Angular/TypeScript e bibliotecas de componentes. Essa diversidade fortalece a análise em relação ao conjunto original, mas também evidencia que o comportamento do UAES pode variar significativamente conforme a estrutura do DOM, a presença de locators

semanticamente distinguíveis e a aderência a boas práticas de desenvolvimento.

Outra ameaça diz respeito ao uso de 79,81% como referência histórica fixa. O intervalo de confiança da replicação incorpora a variabilidade entre as 20 aplicações atuais, mas não a incerteza associada à estimativa do estudo original. Portanto, a comparação indica que o resultado atual ficou abaixo da estimativa publicada, mas não constitui uma comparação entre duas amostras independentes com suas respectivas incertezas.

Por fim, a validação dos elementos *new* e *missed* representa uma ameaça adicional. A análise quantitativa indica divergências entre UAES e Markup, mas a interpretação completa dessas divergências exige inspeção qualitativa dos elementos. Essa inspeção é necessária para verificar se os elementos *new* são de fato acionáveis e relevantes para teste, bem como para identificar as causas dos elementos *missed*, como ausência de atributos distinguíveis, repetição de textos, uso de componentes encapsulados ou estruturas DOM pouco semânticas.

5 Conclusões

Este trabalho apresentou uma replicação experimental estendida do estudo sobre o Unique Actionable Elements Search (UAES), uma abordagem automática para descoberta de elementos acionáveis em aplicações web. A replicação manteve a comparação com a abordagem Markup, utilizada como baseline no estudo original, mas ampliou o conjunto experimental para 20 aplicações open-source com maior diversidade tecnológica e estrutural.

Os resultados mostram que o UAES recuperou 55,32% dos elementos identificados pelo Markup, considerando a métrica *same/(same + missed)*. O intervalo de confiança de 95%, estimado por bootstrap agrupado por aplicação, ficou entre 39,03% e 68,28%. A estimativa foi 24,49 pontos percentuais inferior aos 79,81% reportados no estudo original, e essa referência histórica não esteve contida no intervalo da replicação. Tomando-a como referência fixa, os dados não sustentaram H_0 .

A análise por aplicação mostrou elevada heterogeneidade: a média foi de 63,74%, a mediana de 70,51%, o intervalo interquartil de 40,41% a 85,13% e a amplitude de 10,83% a 100%. Assim, o resultado original não se reproduziu com a mesma magnitude no conjunto ampliado, e o valor agregado não representa uniformemente o comportamento observado em todas as aplicações.

Esta replicação reavalia o UAES em uma amostra maior e evidencia variação substancial entre projetos open-source. Características como qualidade semântica do DOM, presença de atributos acessíveis, textos únicos, componentes encapsulados e locators repetidos constituem explicações plausíveis para essa variação, dado o funcionamento do UAES, mas seus efeitos individuais não foram isolados pelo desenho experimental. Por isso, os resultados não sustentam conclusões causais sobre tecnologias ou padrões de implementação específicos.

Apesar da menor recuperação do baseline Markup, os resultados são promissores sob a perspectiva de aplicabilidade. O UAES foi utilizado sem adaptações para as stacks e bibliotecas presentes nas 20 aplicações, um conjunto mais heterogêneo que os cenários tecnológicos predominantes no estudo original, e ainda recuperou mais da metade do baseline. Além disso, identificou 969 candidatos

classificados como *new*, reforçando seu potencial para revelar elementos não contemplados pelo Markup. A confirmação da validade e da relevância funcional desses candidatos permanece necessária para quantificar com precisão esse benefício complementar.

Como trabalhos futuros, recomenda-se realizar a validação manual sistemática dos elementos *new* e *missed*, investigar qualitativamente as aplicações com pior desempenho, incluir métricas de cobertura e detecção de falhas, e avaliar melhorias no UAES para lidar com bibliotecas visuais modernas, componentes reutilizáveis e padrões de DOM menos semânticos. Também seria relevante ampliar o número de aplicações avaliadas, especialmente em grupos tecnológicos específicos, para investigar com maior precisão quais características de implementação favorecem ou dificultam a descoberta automática de elementos acionáveis. Quanto à marcação assistida por LLM, trabalhos futuros devem aperfeiçoar o processo de geração e validação das marcações, refinando o prompt, automatizando a verificação dos atributos no DOM e avaliando a consistência dos resultados entre diferentes modelos e execuções.

Referências

- [1] Izzeddin Gur, Ofir Nachum, Yingjie Miao, Mustafa Safdari, Austin Huang, Aakanksha Chowdhery, Sharan Narang, Noah Fiedel, and Aleksandra Faust. 2023. Understanding HTML with Large Language Models. *arXiv preprint arXiv:2210.03945* (2023).
- [2] William G. J. Halfond and Alessandro Orso. 2007. Improving Test Case Generation for Web Applications Using Automated Interface Discovery. In *Proceedings of the 6th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering*. ACM, 145–154. doi:10.1145/1287624.1287646
- [3] Thiago Santos de Moura, Everton L. G. Alves, Hugo Feitosa de Figueirêdo, and Cláudio de Souza Baptista. 2023. Cytetion: Automated GUI Testing for Web Applications. In *Proceedings of the XXXVII Brazilian Symposium on Software Engineering*. ACM, 388–397. doi:10.1145/3613372.3613408
- [4] Thiago Santos de Moura, Francisco Igor de Lima Mendes, Everton L. G. Alves, Ismael Raimundo da Silva Neto, and Claudio de Souza Baptista. 2024. An Automatic Approach for Uniquely Discovering Actionable Elements for Systematic GUI Testing in Web Applications. In *2024 IEEE 24th International Conference on Software Quality, Reliability and Security (QRS)*. IEEE. doi:10.1109/QRS62785.2024.00012
- [5] Michel Nass, Emil Alégroth, and Robert Feldt. 2021. Why Many Challenges with GUI Test Automation (Will) Remain. *Information and Software Technology* 138 (2021), 106625. doi:10.1016/j.infsof.2021.106625